SoftNexis

Task 3: Database Integration with JDBC

Project Component: Persistent Contact Manager

Core Description

Upgrade the Servlet/JSP contact manager (Task 2) to a database-backed application using JDBC. Replace temporary in-memory storage with PostgreSQL/MySQL for persistent data management, while adding search capabilities and optimizing database connections.

Detailed Responsibilities

1. Database Schema Design

- ✅ Create normalized `contacts` table:

```
CREATE TABLE contacts (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  phone VARCHAR(20),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

-);
- ✅ Add indexes for search optimization:

```
CREATE INDEX idx_contacts_name ON contacts(name);
```

- `CREATE INDEX idx_contacts_email ON contacts(email);`

2. JDBC Implementation

- ✅ Develop `ContactDAO.java` (Data Access Object):

```
public interface ContactDAO {
    void addContact(Contact contact) throws SQLException;
    List<Contact> getAllContacts() throws SQLException;
    List<Contact> searchContacts(String keyword) throws SQLException;
```

- }
- ✅ Implement CRUD operations in `ContactDAOImpl.java`:

```
public class ContactDAOImpl implements ContactDAO {
    private final DataSource dataSource; // Injected via constructor

    @Override
    public void addContact(Contact contact) throws SQLException {
        String sql = "INSERT INTO contacts (name, email, phone) VALUES (?, ?, ?)";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, contact.getName());
            stmt.setString(2, contact.getEmail());
            stmt.setString(3, contact.getPhone());
            stmt.executeUpdate();
        }
    }
```

- }

3. Connection Pooling

- ✅ Configure HikariCP (or Tomcat JDBC Pool):

```
<!-- META-INF/context.xml -->
<Resource name="jdbc/ContactDB"
          auth="Container"
          type="javax.sql.DataSource"
          factory="com.zaxxer.hikari.HikariJNDIFactory"
          jdbcUrl="jdbc:postgresql://localhost:5432/contactdb"
          username="admin"
          password="secret"
          maximumPoolSize="20"
```

- `          connectionTimeout="30000"/>`
- ✅ Initialize pool in `DBUtil.java`:

```
public class DBUtil {
    private static DataSource dataSource;
```

```
    public static DataSource getDataSource() {
        if (dataSource == null) {
            Context ctx = new InitialContext();
            dataSource = (DataSource) ctx.lookup("java:comp/env/jdbc/ContactDB");
        }
        return dataSource;
    }
```

- }

4. Search Functionality

- ✅ Add search form in `contact-list.jsp`:

```
<form action="contacts" method="get">
  <input type="text" name="search" placeholder="Search contacts...">
  <button type="submit">Search</button>
```

- </form>
- ✅ Implement search in `ContactServlet`:

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) {
    String searchTerm = request.getParameter("search");
    List<Contact> contacts = (searchTerm != null)
        ? contactDAO.searchContacts(searchTerm)
        : contactDAO.getAllContacts();
    request.setAttribute("contacts", contacts);
    // Forward to JSP
```

- }

Technical Specifications

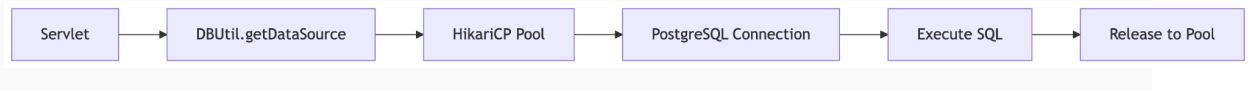
Component	Technology	Responsibility
Database	PostgreSQL/MySQL	Persistent storage with ACID compliance

Connection Pool	HikariCP	Efficient database connection management
Data Access Layer	JDBC + DAO Pattern	Abstract database operations from business logic
Search	SQL LIKE + Indexing	Case-insensitive partial match queries

---

Key Features to Implement

1. Database Connection Lifecycle:



2. Search Algorithm:

```
SELECT * FROM contacts
WHERE LOWER(name) LIKE LOWER('%' || ? || '%')
```

3. `    OR LOWER(email) LIKE LOWER('%' || ? || '%')`
4. Error Handling:
    - SQLException logging
    - Connection timeout recovery
    - Graceful "Service Unavailable" page

Performance Optimization

Technique	Implementation
Prepared Statements	Prevent SQL injection + reuse execution plans

Batch Inserts	`addBatch()` for bulk operations
Connection Validation	`connectionTestQuery = SELECT 1`
Pool Sizing	`maximumPoolSize=20` (adjust based on workload)

Validation Enhancements

1. Database Constraints:

```
ALTER TABLE contacts ADD CONSTRAINT email_format
```

2. `CHECK (email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$');`
3. Duplicate Entry Handling:

```
try {
    contactDAO.addContact(contact);
} catch (SQLException e) {
    if (e.getSQLState().equals("23505")) { // Unique violation
        errors.put("email", "Email already exists");
    }
```

4. `}`

Migration Strategy

1. Data Migration:
    - Export in-memory contacts to CSV
    - Import CSV to database using `COPY` command
2. Schema Versioning:
    - Use Flyway/Liquibase for future migrations

Success Metrics

- ✅ Contacts persist after server restart
- ✅ Search returns results in <500ms (1000+ records)
- ✅ Connection pool serves 50+ concurrent users
- ✅ No SQL injection vulnerabilities
- ✅ Database handles validation failures gracefully

Pro Tip: Use `EXPLAIN ANALYZE` in PostgreSQL to optimize slow queries!

Next Evolution: In Task 4, this JDBC layer will be replaced with Spring Data JPA.